

The image features a background of a classical building facade with columns and arches, likely the main building of the University of Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'U' and 'W' shape. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a larger, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

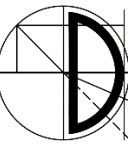
Grundlagen der Programmierung

Kapitel 3: Einfache Datentypen

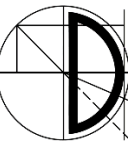
Prof. Dr. Samuel Kounev, Martin Sträßer M.Sc., Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

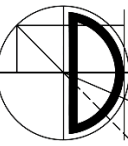
- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- Ausdrücke und Operatoren
- Auswertungsreihenfolge
- Konversionsregeln
- Konstanten



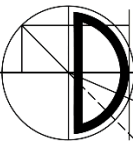
- Jedem deklarierten Bezeichner muss genau ein Typ zugeordnet sein
- Dadurch wird implizit festgelegt:
 - wie viel Speicherplatz zu reservieren ist
 - welche Werte dem jeweiligen Speicherinhalt entsprechen
 - welche Operationen für den Bezeichner definiert sind



- Mit jedem Typ ist ein **Wertebereich (domain)** festgelegt
 - Menge der Werte, welche eine Variable des Typs annehmen kann
- Java/C sind Sprachen mit strenger **Typprüfung (type checking)**
 - Jeder Variable und jedem Ausdruck ist ein zur Übersetzungszeit bekannter Typ zugeordnet
- Generell wird zw. “strongly typed” und “weakly typed” Programmiersprachen unterschieden
 - Keine einheitliche Definition
 - Grob gesagt: Viele Fehler treten bei Sprachen mit strenger Typprüfung bereits beim Übersetzen des Quellcodes auf, während bei schwach typisierten Sprachen viele Fehler erst zur Laufzeit auftreten

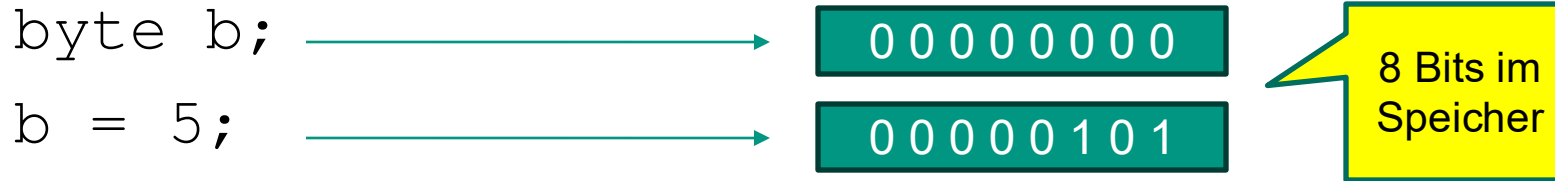


- Java unterscheidet zwei Gruppen von Datentypen
- **Einfache (elementare) Datentypen** (primitive data types)
 - Der logische Typ `boolean`
 - Ganzzahlige Typen (mit Vorzeichen): `byte`, `short`, `int`, `long`
 - Zeichentyp (ganzzahliger Typ ohne Vorzeichen): `char`
 - Gleitkommatypen: `float`, `double`
- **Referenzdatentypen** (reference types)
 - Klassen, Interfaces und Felder (später im Kurs)



■ Einfache Datentypen

- Variable ist mit Adresse im Speicher assoziiert, in der auch direkt der Wert der Variablen gespeichert ist
- Größe der Variable im Speicher ist festgelegt (z.B. `byte` hat 8 Bits)



■ Referenzdatentypen

- Variable ist mit Adresse im Speicher assoziiert, an der die (Start-)Adresse eines größeren Speicherbereichs gespeichert ist. Die eigentlichen Werte liegen an diesem referenzierten Speicherbereichs.
- Erlaubt den Zugriff auf „größere“ Daten über eine Variable





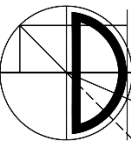
Datentyp	Kleinster Wert	Größter Wert	Länge
byte	-128	127	8 Bits
short	-32768	32767	16 Bits
int	-2147483648	2147483647	32 Bits
long	-9223372036854775808	9223372036854775807	64 Bits

- Ganzzahlige Literalkonstanten
 - Für `byte`, `short` und `int` Ziffernfolgen der Form: `8` `123` `16787`
 - Für `long` Ziffernfolgen mit `l` oder `L` am Ende: `17L` `29874987349L`
 - Alternative Zahlendarstellungen möglich: Binär (z.B. `0b101`), Oktal (z.B. `017`), Hexadezimal (z.B. `0xF1F`)



Datentyp	Kleinster positiver Wert	Größter positiver Wert	Länge
float	~1.4E-45	~3.40282347E38	32 Bits
double	~4.9E-324	~1.79769313486E308	64 Bits

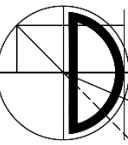
- Java realisiert den Standard IEEE 754 mit 4 byte für einfache Genauigkeit (float) und 8 byte für doppelte Genauigkeit (double)
- In C/C++ ist die Größe der Datentypen plattformabhängig
- Gleitkomma-Literalkonstanten
 - Bestehen aus Mantisse und optionalem Exponentialanteil: `1.23e7`
 - Für `float` muss das Suffix `f` angehängt werden: `1.23e7f`



- Typname `boolean` (Java)
- Typname `bool` (C++)
- Länge ist typischerweise 1 Byte (8 Bits)
- Literalkonstanten: `false` und `true`

In C gibt es generell keinen Typ „boolean“.

Es wird meistens integer verwendet (0 = false, 1 = true).



- Stellt einzelne Zeichen dar
- Typname `char`
- Länge ist 16 Bits in Java und typischerweise 8 Bits in C/C++
- Literalkonstanten werden in Hochkommas (nicht zu verwechseln mit Anführungszeichen!) eingeschlossen

`'a'` `'x'` `'+'` `'ä'`

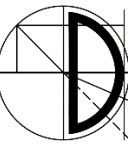
`'\n'` Ersatzdarstellung für 'Zeilenende'

`'\t'` Ersatzdarstellung für 'Tabulator'

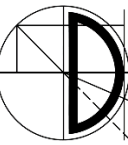
`'\''` Ersatzdarstellung für `' '` (da unzulässig)

`'\"'` Ersatzdarstellung für `'\"'` (aber zulässig)

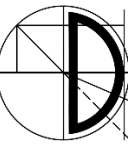
`'\\'` Ersatzdarstellung für `'\'` (da unzulässig)



- Unicode-Zeichen entstehen aus dem Präfix `\u` und einer 4-stelligen Hexadezimalzahl
- Beispiel: `'\u007B'` entspricht dem Zeichen `'{'`



- Datentyp: `String`
 - Häufig verwendet, aber kein einfacher Datentyp
 - Literalkonstanten sind in Doppelhochkommas (Anführungszeichen) eingeschlossen (z.B. `"HalliHallo"`)
 - Können auch Ersatzdarstellungen für spezielle Zeichen enthalten (z.B. `"Ach\ndu\nlieber\n\"Himmel\""`)
- In C gibt es keinen Typ `String`!
 - Es werden Felder vom Typ `char` verwendet



- Bisher haben wir nur **lokale Variablen** in der Methode `main` verwendet. Für diese muss stets explizit ein Wert festgelegt werden.
- In vielen Fällen wird später Variablen automatisch ein Wert zugewiesen, auch wenn keine explizite Wertzuweisung erfolgt - siehe später **Instanzvariablen**
- Solche Werte werden **Default-Werte** genannt. Für die bisher behandelten Typen sind die folgenden Default-Werte festgelegt:
 - 0 bei numerischen Datentypen (ganzzahlig 0, Fließkomma 0.0)
 - `false` bei `boolean`
 - `'\u0000'` bei `char`

- Bemerkung: In C gibt es keine Default-Werte!

- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- Ausdrücke und Operatoren
- Auswertungsreihenfolge
- Konversionsregeln
- Konstanten



■ Beispielhafte Initialisierungen

```
byte b = 5;
```

```
short s = 50;
```

```
int i = 500;
```

```
long l = 5000L;
```

```
float f = 1.5f;
```

```
double d = 2.5;
```

```
char c = 'a';
```

```
boolean x = true;
```

Welche der nachfolgenden Zuweisungen ist (un-)zulässig?

```
i = b;
```

```
f = l;
```

```
i = c;
```

```
s = f;
```

```
i = l;
```

```
i = x;
```

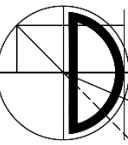


■ Beispielhafte Initialisierungen

```
byte b = 5;  
short s = 50;  
int i = 500;  
long l = 5000L;  
float f = 1.5f;  
double d = 2.5;  
char c = 'a';  
boolean x = true;
```

Welche der nachfolgenden Zuweisungen ist (un-)zulässig?

```
i = b; // erlaubt  
f = l; // erlaubt, aber Datenverlust möglich  
i = c; // erlaubt  
s = f; // unzulässig  
i = l; // unzulässig  
i = x; // unzulässig
```



Typen kann man sich als Becher oder auch als Schablonen vorstellen.

Aber: Wenn Sie den Inhalt eines großen Bechers in einen kleinen schütten, so geht oft etwas verloren.

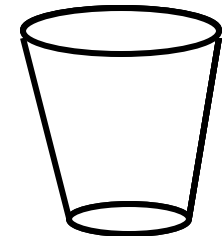
Die Größe eines Bechers bezieht sich auf den Wertebereich (value range).

Beachten Sie, dass `int` und `float` beide 32 Bits haben und `long` und `double` beide 64 Bits haben. **Die Darstellung der betreffenden Zahlen ist aber eine andere** – Idee Schablone

byte
8 Bits



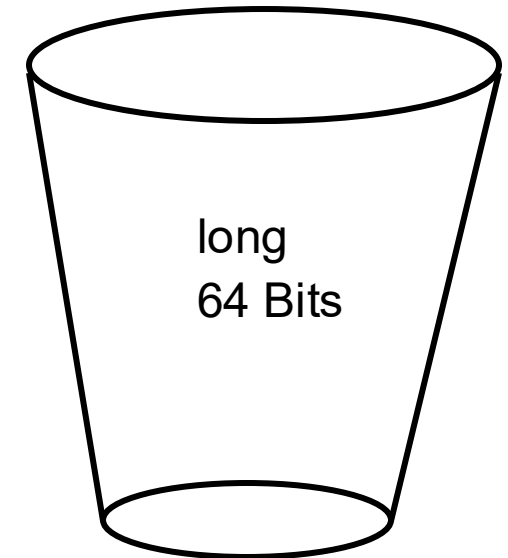
short
16 Bits

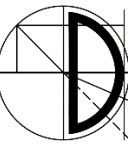


int
32 Bits



long
64 Bits

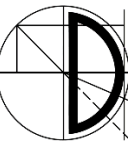




1. Automatische (implizite) Typumwandlung

Automatische (implizite) Typumwandlungen in der Form $y = x$; sind immer dann möglich, wenn der Wertebereich des Datentyps von y mindestens so groß ist wie der von x , d.h. der Wertebereich von x muss eine Teilmenge vom Wertebereich von y sein.

Das gilt auch für Umwandlungen von `int` nach `float` (je 32 Bits) und von `long` nach `double` (je 64 Bits). Hier ist aber aufgrund der unterschiedlichen Darstellung Datenverlust möglich.



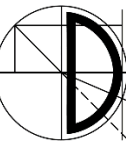
2. Explizite Typumwandlung

Explizite Typumwandlung werden durch Voranstellen des eingeklammerten Namens des Zieldatentyps realisiert. Eventueller Datenverlust wird explizit in Kauf genommen.

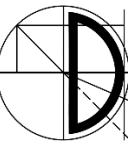
```
long l = 5000L;  
float f = 1.5f;  
s = (short) f;  
i = (int) l;
```

Grundbedingung für jede Typumwandlung ist allerdings, dass die Datentypen **kompatibel** sein müssen, so ist z.B. jede Typumwandlung von und nach `boolean` unzulässig.

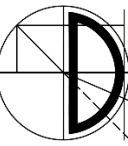
- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- **Ausdrücke und Operatoren**
- Auswertungsreihenfolge
- Konversionsregeln
- Konstanten



- Beispiel für einen Ausdruck: $3.5 * x - 2 / (4 + y)$
- Ein Ausdruck setzt sich zusammen aus:
 - Operanden (Konstanten, Variablen, Methodenaufrufe)
 - Operatoren (+, -, *, ...)
- Ein Ausdruck liefert stets einen Wert!



- Abhängig vom *Typ der Operanden* sind nur bestimmte Operatoren zulässig, z.B.
 - arithmetische Operatoren für die *numerischen Datentypen*
 - Vergleichs-Operatoren für die *numerischen Datentypen*
 - logische Operatoren für den *logischen Datentyp*
 - der Operator `+` für Zeichenketten (Typ `String`)
- Man unterscheidet
 - einstellige (monadische, unäre) Operatoren (z.B. `+`, `-`, `++`, `--`)
 - zweistellige (dyadische, binäre) Operatoren (z.B. `+`, `*`, `&&`, `<`, `<=`)
 - dreistellige (triadische, ternäre) Operatoren (nur einer: `? :`)



- **Elementare (also einfachste) Ausdrücke** sind

1. nur ein Operand und kein Operator

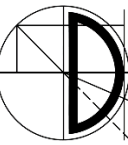
- z.B.
 - `x`
 - `1.74`
 - `Math.PI`

2. nur ein einfacher Methodenaufruf

- z.B.
 - `Math.sin(x)`
 - `Math.sqrt(2)`

Elementarfunktionen der Mathematik
(`exp`, `sin`, `cos`,...) sowie Näherungen
für die Konstanten e und π sind in
Java in die **Klasse** `Math` ausgelagert

- Die Argumente von Methodenaufrufen können natürlich selbst wieder Ausdrücke sein



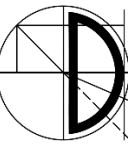
- + Identität (ohne Wirkung)
- Negation
- ++ Inkrement (Erhöhung um 1)
- Dekrement (Erniedrigung um 1)

Verwendung von +, -, ++ und -- in Präfix-Notation (vorgestellt)

Verwendung von ++ und -- in Postfix-Notation (nachgestellt)

Beispiele für `int x = 5`

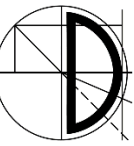
Ausdruck	+x	-x	++x	--x	x++	x--
x vor der Auswertung	5	5	5	5	5	5
Wert des Ausdrucks	5	-5	6	4	5	5
x nach der Auswertung	5	5	6	4	6	4



+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Rest/Modulo (bei Division mit ganzzahligem Ergebnis)

Beispiele

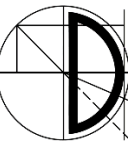
```
int i = 7 / 3;  
int j = 7 % 3;  
double d = 8.2 / 4.0;  
double e = 8.2 % 4.0;  
int x = 5; int y = (x++) + (x++);
```



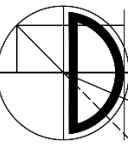
+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Rest/Modulo (bei Division mit ganzzahligem Ergebnis)

Beispiele

```
int i = 7 / 3;           // i = 2
int j = 7 % 3;           // j = 1
double d = 8.2 / 4.0;    // d = 2.05
double e = 8.2 % 4.0;    // e = 0.2 (=0.1999999999...)
int x = 5; int y = (x++) + (x++); // y = 11      x = 7
```



- Der Operator + für Strings
 - bewirkt das Aneinanderhängen von Zeichenketten (**concatenation**)
`String s = "Halli" + "Hallo"; // s ist "HalliHallo"`
- Zusammengesetzte Zuweisungsoperatoren (**compound assignment operators**)
 - zur Abkürzung von Zuweisungen mit einfachen Operationen
die Anweisung `a += b;` // entspricht Anweisung `a = a + b;`
die Anweisung `x *= y;` // entspricht Anweisung `x = x * y;`
 - Vorsicht: Die Lesbarkeit eines Programms verschlechtert sich!



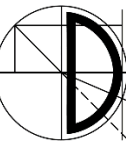
- < Test auf "kleiner"
- > Test auf "größer"
- <= Test auf "kleiner oder gleich"
- >= Test auf "größer oder gleich"
- == Test auf "gleich"
- != Test auf "ungleich"

Beispiel:

Java

```
int x = 2;  
boolean b = (x <= 5);  
System.out.println("5.2 <= 4 is " + (5.2 <= 4));
```

Ausgabe: 5.2 <= 4 is false



&	logisches UND
	logisches ODER
^	logisches exklusives ODER (entweder oder)
& &	logisches UND
	logisches ODER
!	logische NEGATION (Nicht)

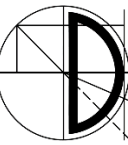
Doppelzeichen && und || ermöglichen vorzeitigen Abbruch der Ausdrucks-Auswertung, sobald das Ergebnis feststeht

```
int a = 0;
```

```
int b = 10;
```

```
boolean wahrOderFalsch = (a != 0) & (b/a > 5);
```

Java



Seien A und B Ausdrücke vom Typ `boolean`

A	B	A & B
true	true	true
true	false	false
false	true	false
false	false	false

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

A & B : beide Ausdrücke A und B werden ausgewertet

A && B : liefert **A** den Wert **false**,

so wird **B nicht mehr ausgewertet**



Seien A und B Ausdrücke vom Typ `boolean`

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

A | B : beide Ausdrücke A und B werden ausgewertet

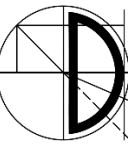
A || B : liefert **A** den Wert **true**,
so wird **B nicht mehr ausgewertet**



Seien A und B Ausdrücke vom Typ `boolean`

A	B	$A \wedge B$
true	true	false
true	false	true
false	true	true
false	false	false

A	$\neg A$
true	false
false	true

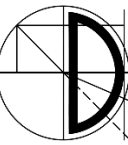


- ~ Bitweise NEGATION
- & Bitweises UND
- | Bitweises ODER
- ^ Bitweises exklusives ODER

x	~x
0	1
1	0

x	y	x & y	x y	x ^ y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- Lassen ganzzahlige Operanden zu, arbeiten aber nicht mit dem ganzzahligen Wert der Operanden, sondern nur mit deren Bits
- Bei den bitweisen Verknüpfungen zweier ganzzahliger Operanden werden diese Bitoperationen auf mehrere Bits (Stelle für Stelle) gleichzeitig angewendet



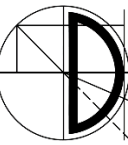
Beispiele:

```
byte a, b;  
a = 11;           // 00001011 = 11  
b = 13;           // 00001101 = 13  
System.out.println(~a); // 11110100 = -12  
System.out.println(a&b); // 00001001 = 9  
System.out.println(a|b); // 00001111 = 15  
System.out.println(a^b); // 00000110 = 6
```



?

Grund: Zweierkomplement-Darstellung (siehe Übung)
 $-128 + 64 + 32 + 16 + 4 = -12$



`a << b`

Schiebt die Bits in `a` um `b` Stellen nach links und füllt mit 0-bits auf

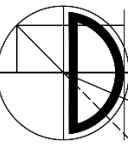
`a >> b`

Schiebt die Bits in `a` um `b` Stellen nach rechts und füllt mit dem höchsten Bit von `a` auf (bewahrt Vorzeichen)

`a >>> b`

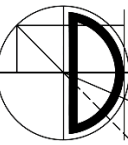
Schiebt die Bits in `a` um `b` Stellen nach rechts und füllt mit 0-bits auf

Das Schieben um eine Stelle nach links bzw. rechts entspricht dabei einer Multiplikation bzw. Division des Wertes mit bzw. durch 2



Beispiele:

```
byte b = -4;           // 11111100
b << 1                 // 11111000 (= -8)
b >> 1                 // 11111110 (= -2)
b >>> 1                // 01111110 (= 126)
```



- Normalerweise treten Wertzuweisungen als Anweisung auf:

$a = b + 3 / c;$

- In Java: Die Zuweisung ist selbst wieder ein Ausdruck. Der Wert eines Zuweisungsausdrucks ist der neue Wert der jeweils linken Seite.

- Daher ist folgende **Mehrfachzuweisung** erlaubt:

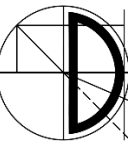
$d = e = f = g = h = a = b + 3 / c;$

- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- Ausdrücke und Operatoren
- **Auswertungsreihenfolge**
- Konversionsregeln
- Konstanten

Operator-Prioritätsstufen

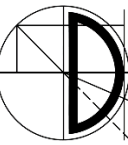
C/C++

Java



Postfixoperatoren	[] . () ++ --	Hohe Priorität Niedrige Priorität
einst. Operatoren	++ -- + - ~ !	
Erzeugung/Cast	new (Typname)	
multiplikativ	* / %	
additiv	+ -	
shift	<< >> >>>	
relational	< > <= >= instanceof	
Gleichheit	== !=	
logisches/bitweises UND	&	
logisches/bitweises XOR	^	
logisches/bitweises ODER		
logisches UND	&&	
logisches ODER		
Bedingung	? :	
Zuweisung	= += -= *= /= %= &= ^= = <<= >>= >>>=	

Merkregel: **einstellig vor multiplikativ vor additiv vor**
vergleichend vor logisch vor zuweisend



- Regeln für die Auswertungsreihenfolge in Ausdrücken
 - Operatoren mit **höherer Priorität** vor Operatoren mit niedrigerer Priorität
 - **mehrere zweistellige** Operatoren mit gleicher Priorität von links nach rechts (**links-assoziativ**)
 - **mehrere einstellige** Operatoren und Zuweisungsoperatoren von rechts nach links (**rechts-assoziativ**)
 - **elementare Ausdrücke** und **Ausdrücke in Klammern** vor den Operatoren, die diese weiterverknüpfen
 - **linker Operanden-Ausdruck** immer vor dem rechten Operanden-Ausdruck
 - Beispiel: Wie wird der folgende Ausdruck ausgewertet?

`Math.sqrt(3.5+x) * 5 / 3 - (x+10) * (x-4.1) < 0`

②

①

③

④

⑧

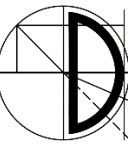
⑤

⑦

⑥

⑨

- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- Ausdrücke und Operatoren
- Auswertungsreihenfolge
- Konversionsregeln
- Konstanten



■ String-Konversion

- Ist `s` vom Typ `String` und `x` von einem beliebigen anderen Typ, so wird bei Auswertung des Operators `+` in

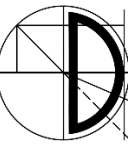
`x + s` oder `s + x`

`x` implizit nach `String` gewandelt

■ Numerische Typ-Konversion

- Werden zweistellige Operatoren (z.B. `+`, `-`, `*`, `<`, `==`) auf numerische Operanden angewandt, so wird wie folgt vorgegangen:
 - Falls ein Operand vom Typ `double` ist, so wird der andere nach `double` konvertiert
 - Andernfalls, falls ein Operand vom Typ `float` ist, so wird der andere nach `float` konvertiert
 - Andernfalls, falls ein Operand vom Typ `long` ist, so wird der andere nach `long` konvertiert
 - Andernfalls werden beide Operanden nach `int` konvertiert

- Datentypen und ihre Literale
- Zuweisungen und Typumwandlungen
- Ausdrücke und Operatoren
- Auswertungsreihenfolge
- Konversionsregeln
- Konstanten



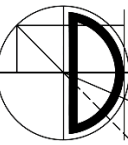
- Syntaxregel für `final`-Variablen / symbolische Konstanten

```
final <VARIABLENTYP> <VARIABLENBEZEICHNER> = <AUSDRUCK>;
```

- Beispiel:

```
final double WECHSELKURS = 0.723;
```

- Unzulässig wäre danach dann: `WECHSELKURS = 1.47;`
- Konstanten darf nur einmal ein Wert zugewiesen werden



- Syntaxregel für **const**-Variablen / symbolische Konstanten

```
const <VARIABLENTYP> <VARIABLENBEZEICHNER> = <AUSDRUCK>;  
oder  
<VARIABLENTYP> const <VARIABLENBEZEICHNER> = <AUSDRUCK>;
```

- Beispiel:

```
const double WECHSELKURS = 0.723;
```

- Unzulässig wäre danach dann: **WECHSELKURS = 1.47;**

Schon gewusst, dass in der Programmiersprache Python der ganzzahlige Datentyp `int` „keine“ obere Wertegrenze hat?

Da Python nicht zwischen unterschiedlichen ganzzahligen Datentypen unterscheidet, gibt es auch keine festen Wertegrenzen. Der Python Interpreter legt allerdings fest, wie lang das längste zusammenhängende Speicherwort sein kann.



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a large, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit